



SHASU VATHANAN

SHASU VATHANAN - GEN AI - PRODUCT MANAGER

SHASUVATHANAN.COM · [linkedin.com/in/shasuvathanan](https://www.linkedin.com/in/shasuvathanan)

GEN AI ARCHITECT JOURNEY · PHASE-005

LLM Foundations and Practical Tools

From what the three letters in LLM actually mean, to a model running locally on your own laptop — ten topics, end to end.

10 TOPICS

OPEN SOURCE

CLOSED SOURCE

BENCHMARKS

HANDS-ON

TOPICS

10

THEMES

4

COMPANION DECK

102 slides

HANDS-ON

Ollama + HF

INSIDE THIS DOCUMENT

01 LLM and Its Types

02 Open-Source LLM vs Cloud LLM

03 Closed-Source LLM

04 How to Purchase API Keys

05 LLM Benchmarks

06 Essential AI Tools

07 Hugging Face Spaces

08 Pulling a Model from Hugging Face

09 Running a Model Using Ollama

10 Ollama — Hands-On Practice

Shasu Vathanan - GEN AI - Product Manager

SHASUVATHANAN.COM · <https://www.linkedin.com/in/shasuvathanan>

CORAL RED #FF4A62 → #FF5A45

DEEP BLUE #031273



How to Use This Document

This is the written companion to **LLM-Foundations-and-Practical-Tools.pptx**. The deck is the visual walkthrough; this document is the reference you come back to for the commands, the prices, the numbers and the checklists. Topic numbers, sub-topic numbers and terminology are identical across the deck, this PDF and the phase **README.md**, so any one of the three can be read on its own.

01

Read one topic at a time

Each topic opens with what it covers, then works through its numbered sub-topics in order. Nothing later depends on anything you have skipped.

02

Do the task before moving on

Topics 2, 3, 4, 7 and 10 each end in a practical task. They are short, and they are where the understanding actually forms.

03

Use the deck for the visuals

Diagrams, comparisons and flows are laid out slide by slide in the companion PowerPoint. Same numbering, same order.

04

Come back to this as reference

Commands, pricing, context-window limits and checklists are all here and are meant to be looked up rather than memorised.

The Four Themes

The ten topics group into four themes. Each theme answers one question, and together they take you from not knowing what an LLM is to having one running on your own machine.

Foundations

Topics 1–2

The closed path

Topics 3–5

The toolkit

Topics 6–7

The open path

Topics 8–10

The Ten Topics

#	Topic	What it covers
01	LLM and Its Types	What the three letters mean, and how a prompt becomes an answer
02	Open-Source LLM vs Cloud LLM	Two families of model — and how to read a model card properly
03	Closed-Source LLM	The four major players, their platforms, and exactly what you pay
04	How to Purchase API Keys for Closed LLMs	One key, ten dollars, and the setting that stops it running away
05	LLM Benchmarks	How models are measured, and how to turn a score into a decision
06	Essential AI Tools	Three categories of tool, and the small free set actually used
07	Hugging Face Spaces	The AI world's social platform, and the free apps running inside it



#	Topic	What it covers
08	Pulling an Open-Source Model from Hugging Face	Method one — pull a model with Python and run it on your machine
09	Running an Open-Source Model Using Ollama	Method two — install, pull, run. The easiest local path there is
10	Open-Source Model with Ollama — Hands-On Practice	Install, pull Qwen2.5 0.5B, chat with it, and capture the proof

What You Will Be Able to Do

BY THE END YOU CAN EXPLAIN

The concepts, in your own words

- How an LLM turns a prompt into an answer, in five steps
- What a token is, and why it is the unit you are billed on
- What embeddings are and why a model needs them
- The difference between LLM, LMM, SLM and VLM
- What a context window is, and what it is not

BY THE END YOU CAN DO

The practical work

- Read any model card and pull out the four things that matter
- Buy an API key safely, with auto top-up disabled
- Read a benchmark table and turn it into a model choice
- Pull a model from Hugging Face and run it in Python
- Install Ollama and chat with a local model in two commands



01

FOUNDATIONS

LLM and Its Types

Start with the fundamentals. Once you understand how an LLM works, every term that follows has somewhere to land. This topic covers six sub-topics, from the meaning of the three letters to the full family of model types.

1.1 Breaking Down the Three Letters

Every part of the name tells you something concrete about how the model was built.

LARGE

The scale of the training data

- Trained on massive datasets — 1 trillion data points is now normal
- Earlier generations trained on millions, then billions
- That jump in scale is why models feel so capable today

LANGUAGE + MODEL

The form of the data, and the engine

- Language is not only text — image, audio and video all count
- Any human language works: ask in Tamil, get Tamil back
- Model = a neural-network engine that generates the answer

1.2 How an LLM Actually Works

Five steps, all completed in milliseconds. Learn the flow, not the internal mathematics.

1	Input	You type a prompt — for example, “Write a Python code for sum of two numbers”.
2	Encoder → tokens	Your text is cut into tokens, the unit the model counts.
3	Embeddings	Tokens become mathematical numbers that carry meaning.
4	LLM	Self-attention predicts the next word, again and again.
5	Decoder → output	Embeddings become tokens, and tokens become readable text.

THE SAME FIVE STEPS, EVERYWHERE

This flow runs whether you use ChatGPT, Claude, Gemini or a model on your own laptop. The provider changes; the pipeline does not.



1.3 What Exactly Is a Token?

Tokens are the unit you are billed on — so it pays to know how your sentence is divided.

10

TOKENS

“Write a Python code for sum of two numbers”

5

TOKENS

“White sky is blue” plus a special character

1

TOKEN

Roughly every word before a space

THE SIMPLE RULE

How the split works

- Everything before a space is normally one token
- Special characters are counted separately
- A long word such as “classification” splits into more than one token

WHY IT MATTERS

Where your bill comes from

- “Token usage is high” and “the cost went up” both trace back to this count
- Each model has its own tokenisation algorithm
- You never do this by hand — the system handles it

1.4 Embeddings: Meaning as Mathematics

A machine does not understand natural language or code. It only understands numbers.

Your words

The prompt as you typed it

Tokenised

The sentence split into tokens

Embedded

Each token becomes a mathematical representation

Meaningful

The numbers carry the meaning of word and sentence

DEFINITION

An **embedding** is a mathematical representation of a meaningful word. This is the language the model actually thinks in.

1.5 Attention, Prediction and Decoding

Two ideas explain almost everything you see in a chat window.

NEXT-WORD PREDICTION

The model does not store your data

- Even trained on 1 trillion data points, the data is not stored inside
- What it holds is the ability to predict the next word
- That is why answers stream out word by word, not as a finished paragraph
- Self-attention checks the words already written to keep the answer correct

DECODING

Simply the reverse of the input path

- The generated answer first exists as embeddings
- Embeddings are converted back into tokens
- Tokens are converted back into readable words
- Ask in English → English out. Ask in Tamil → Tamil out.



Architecture families you will hear about

Three names get used constantly. You only need to recognise them — the internals are handled for you.

ENC

Encoder-only

Focused on understanding the input rather than generating long text.

DEC

Decoder-only

The method used for generating answers. ChatGPT, Claude and Grok all sit here.

E+D

Encoder-Decoder

Both stages combined, used where input must be understood and output regenerated.

WORTH KNOWING

Almost every chat model you use day to day is decoder-only.

1.6 The Model Family Tree

Four acronyms cover the model types you will meet. They are the same idea sized and shaped for different jobs.

Type	Name	What it handles	Typical home
LLM	Large Language Model	Large-data text work, the mainstream of the industry	Industry and enterprise deployments
LMM	Large Multimodal Model	Text, image, audio and video generation together	Google Gemini, ChatGPT
SLM	Small Language Model	Smaller data, lighter footprint, installable locally	Your own machine
VLM	Vision Language Model	Object detection and video analysis — eyes for the model	Camera, vision and video use cases

DO NOT MEMORISE THE LIST

New model types keep appearing for new use cases. Understand how they work and the next acronym will explain itself.

Key Takeaway

FLOW

Input → tokens → embeddings →
LLM → decode → output

Five steps, every time, on every provider.

UNIT

Tokens are what you are counted
and charged on

Keep prompts tight and the bill follows.

FAMILY

LLM, LMM, SLM and VLM

The same idea sized and shaped for different jobs.

**02**

FOUNDATIONS

Open-Source LLM vs Cloud LLM

Understand this one distinction and a great deal of the confusion around LLMs simply disappears. Every model you will ever use falls into one of two families.

2.1 Two Families of LLM

OPEN SOURCE*The model file is yours*

- You receive the actual model files
- Run it on your own laptop or your own server
- You are free to change the model and use it
- Covered hands-on in Topics 8, 9 and 10

CLOSED SOURCE*Usage only, never the file*

- The provider lets you use the model — not own it
- No model files are given to you
- You pay in order to use it
- You cannot make changes yourself

2.2 Open Source vs Cloud LLM, Side by Side

The same six questions asked of both families.

	Open-source LLM	Cloud / closed LLM
Access	Model files are downloaded to you	Usage rights only, through the provider
Deployment	Your laptop, your server, your VM	The provider's cloud infrastructure
Cost	No API cost once the model is downloaded	Pay per million input and output tokens
Customisation	Free to modify and adapt the model	No changes possible on your side
Privacy	Data stays on the machine you control	Requests travel to the provider
Infrastructure	Your RAM and GPU decide speed and quality	Provider-scale hardware, nothing to manage

BOTH ARE VALID

Neither family is the “right” one. Choose by use case — privacy and cost push you open; scale and convenience push you closed.

2.3 Where Open-Source Models Live

Two platforms cover almost everything you will need.



OLLAMA

ollama.com

Open the Models section and every model listed there is open source and directly downloadable. This is the easy path, covered in Topic 9.

HUGGING FACE

huggingface.co

Models, Datasets and Spaces — the same open models, plus a great deal more built around them. Covered in Topics 7 and 8.

FILTER BY SIZE

Both platforms let you filter by parameter count — from 0.5 billion up to models in the hundreds of billions. Small models belong on your laptop; large ones belong on a server or VM.

2.4 The Closed-Source Players

Many companies exist, but four are currently leading the industry.

OA

OpenAI

The parent company of ChatGPT. Player number one in the closed-source race.

AN

Anthropic

The company behind Claude.

XAI

xAI

The company behind Grok.

G

Google

Gemini, from Google DeepMind.

FRONTIER MODELS

Each of these labs ships frontier models — models proven on benchmark reports to outperform a human at a given task. Topic 5 covers how to read those reports.

2.5 How to Read a Model Card

Four things to note down for every model you evaluate. This is a skill, and you build it by repeating it.

1

Capabilities

Vision — analyses an image or video. Tools — calls Gmail, Slack and others to complete a task. Thinking — reasons before generating rather than answering straight away.

2

Usage

What the model is actually built for — the kind of work where it is worth reaching for.

3

Context window

How much input the model can take in at once, measured in tokens.

4

Size

The parameter count the model was trained with, and the download size on disk.



Context window, explained simply

This is the single most misunderstood number on a model card.

THE DOOR ANALOGY

Why size limits matter

- Picture a house with one small door
- A very large almirah simply will not go through it
- Whatever you send has to fit through that opening
- The context window is that door

IN PRACTICE

Input capacity, in tokens

- A 256K window accepts roughly 2.56 lakh tokens of input
- Send 3 lakh tokens and the model cannot take it in
- Context window = how much input an LLM can accept
- Parameters are the separate measure — the scale it was trained at

Model cards decoded — worked examples

Three real open models read the same way, so the pattern becomes automatic.

Model	Size	Context	What stands out
Kimi K2.7	1.04 trillion parameters	256K tokens	Built on K2.6, tuned for long-horizon coding; 30% lower thinking-token usage
LFM 2.5 8B A1B	8 billion parameters · 5.2 GB	125K tokens	An edge model — built for small devices; text input only
Gemma (E2B)	7.2 GB on disk	See model card	Edge-device family; over 14 million downloads

WORTH PAUSING ON

1.04 trillion parameters available as open source is a genuinely big deal. Weights at that scale used to be closed by default.

2.6 Your Task for This Topic

Pick one open-source model and decode it completely, then write it up.

Choose one open-source model	From Ollama or Hugging Face — any model that interests you
Note the capabilities	Vision, tools, thinking — record all of them
Note the usage	What the model is designed and recommended for
Note the context window	How much input it can understand at once
Note the size	Parameter count and the download size
Note the input types	Text, image, audio, video — which ones it accepts



DEFINITION OF DONE

One row in your sheet, six columns filled. This is the first practical task of the phase.

**03**

THE CLOSED PATH

Closed-Source LLM

You never touch the model. You touch an API — and understanding that ecosystem is what makes you effective with it. This topic is a full tour of the closed-source side: who the players are, how the platforms are laid out, and how you are charged.

3.1 The Closed-Source Ecosystem

1	You / your app	Your application, script or no-code tool sends a prompt.
2	API key	Your paid key authenticates and meters the request.
3	Cloud provider	OpenAI, Anthropic, xAI or Google receives it.
4	Proprietary model	The model runs on their hardware — the files never leave.
5	Response	Generated output returns to your application.

WHY IT IS CALLED CLOSED SOURCE

You get the answer, never the weights. That single sentence is the whole distinction.

3.2 The Players and the People

Company	Led by	Known for
OpenAI	Sam Altman	The parent company of ChatGPT
Anthropic	Dario Amodei	The company behind Claude
xAI	Elon Musk	The company behind Grok
Google	Sundar Pichai	The company behind Gemini

THE RACE YOU KEEP HEARING ABOUT

The “who is first” competition in the news is this race, between these four labs.

3.3 Walking an Actual Platform

The OpenAI platform, step by step. The same shape repeats on every provider.

1	Open platform.openai.com	Search for it and open the platform site.
2	Sign in	This is closed source — nothing is visible until you log in.



3	Search for “models” in the docs	You land in the developer documentation.
4	Read the frontier models list	Each model is listed with its own model ID.
5	Check reasoning types and pricing	Per-model reasoning options and per-million-token rates.
6	Check context, max output and cutoff	The three limits that decide whether a model fits.

3.4 Frontier vs Specialised Models

Every closed-source catalogue splits into these two shelves.

FRONTIER MODELS

The general-purpose flagships

- Built for complex reasoning and best-in-class performance
- Designed to work beyond human level on hard tasks
- The flagship, then a mini and a nano tier beneath it
- This is where the headline benchmark numbers come from

SPECIALISED MODELS

One job, done extremely well

- Image generation
- Real-time speech and translation
- Speech and text-to-speech
- Transcription and other single-purpose endpoints

3.5 What You Actually Pay

Prices are quoted per 1 million tokens. The gap between the flagship and the small models is enormous.

\$5

FLAGSHIP INPUT

per 1 million input tokens

\$30

FLAGSHIP OUTPUT

per 1 million output tokens

\$0.20

NANO INPUT

per 1 million input tokens

INPUT IS THE BIG NUMBER

You send a lot

- Long prompts, documents and context all count as input
- “Write a Python code for sum of two numbers” alone is 10 tokens
- Understanding your input is where the volume goes
- 1 million input tokens on the flagship costs \$5

OUTPUT IS THE SMALL NUMBER

The model sends back less

- Generation is usually far smaller than the input
- You will rarely hit 1 million output tokens
- But it is priced much higher — \$30 per million on the flagship
- So keep prompts tight and pick the smallest model that works



THE WORKING RULE

Output on the flagship is charged at \$30 per million tokens — very high. Throughout this phase, use the mini and nano tiers only.

3.6 The Three Limits That Matter

Read these before you commit a model to any application.

400K – 1M

CONTEXT WINDOW

How much input the model can take in

128K

MAX OUTPUT TOKENS

The ceiling on a single generation

Fixed date

KNOWLEDGE CUTOFF

Nothing after that date is known

1

Context window

Currently up to 1 million tokens on the top models — roughly four times what many open models offer. 400K is already more than enough for most use cases.

2

Max output

128K tokens is the maximum in one generation. Push past it and you get hallucinated output; generate long results in two or three passes instead.

3

Knowledge cutoff

The model only knows data up to its cutoff date. Ask about later events and it cannot answer — which is why web search is bolted on, and why RAG exists.

3.7 The Other Consoles

Same pattern, four more doors. Search the console name, log in, open the models page.

Provider	Console	What you will find
Anthropic	Claude platform / console	The Claude model family, with the full model list after login
xAI	console.x.ai	Grok, a voice API and an image API — notably cheap image generation
Google	ai.google.dev	The Gemini family, context windows past 1 million, capability and cost per model
Moonshot	platform.moonshot / Kimi	Kimi, a developer console and docs, plus the billing unit and pricing page

A PATTERN WORTH NOTICING

Several Chinese labs do both: they release the model file openly *and* sell API access to it. Open weights and a cheap API are not mutually exclusive.



3.8 Your Task for This Topic

Do for closed models exactly what you did for the open one in Topic 2.

Pick five models	Across OpenAI, Anthropic, xAI and Google
Record the capabilities	What each model can actually do
Record the context window	Input capacity in tokens
Record the max output	Generation ceiling per request
Record the cost	Input and output price per million tokens
Record the comparison	One sheet, five models, the same columns

WHY THIS MATTERS

Learning to find capabilities and cost yourself is the real skill being built here. The numbers change every few months; the method does not.



04

THE CLOSED PATH

How to Purchase API Keys for Closed LLMs

A short, practical topic with one clear instruction and one very important warning. One key, ten dollars, and a set of guard rails learned the expensive way.

4.1 Buy One Key. Only One.

Keys are available from OpenAI, Anthropic, xAI, Google and Moonshot. Purchase the OpenAI one.

WHY THIS ONE

Compatibility is the whole reason

- The OpenAI API format has the widest compatibility
- Any framework you pick will plug into it easily
- No-code tools connect to it without friction
- Fewer issues, fewer blockers, more building

THE INSTRUCTION

Stated plainly

- Purchase the OpenAI API key only
- Top it up with \$10 — not more
- Do not jump to other providers while learning
- One key is genuinely enough for everything ahead

4.2 The Budget and the Model Discipline

Two numbers to hold on to for the whole phase.

\$10

TOTAL TOP-UP

That is genuinely all you need to start

mini

FIRST CHOICE

Use the mini model wherever it works

nano

CHEAPEST OPTION

\$0.20 per million input tokens

WHY MINI AND NANO

The default choice

- Every time you develop or implement, reach for mini or nano
- Very low cost with genuinely good intelligence
- Fast, with high reasoning for what they cost

WHERE THE FLAGSHIP FITS

The exception, not the rule

- The top model is very costly for everyday learning
- Save it for work that actually needs it
- Learning use cases run comfortably on the small models

4.3 The Purchase Flow, Step by Step

Follow this order exactly and nothing surprises you later.

1

Open platform.openai.com

Sign in — you will land on the platform page.

2

Search “billing” → Add credits

The credits page is where the top-up happens.



3	Add a payment method	Card name, expiry and security code.
4	Add \$10 in credits	Ten dollars only — no more while you are learning.
5	Disable auto top-up	Find it, switch it off, and confirm it is off.
6	API keys → Create new secret key	Give it a clear name, for example “GenAI learnings”.
7	Copy and store the key safely	This key is now your access to every OpenAI model.

4.4 Payment Cards — What Actually Works

A practical detail that stops many people at the last step.

WHAT WORKS

International-enabled cards

- Foreign / international transactions must be enabled
- Visa and Mastercard are the usual routes
- You only need it to clear a single \$10 charge

WHAT TO EXPECT

Common blockers

- Some debit cards are simply blocked for these payments
- The charge is small — bank-side limits are rarely the issue
- Enter card name, expiry and security code exactly as issued

4.5 The \$250 Lesson — Turn Auto Top-Up Off

THE WARNING

“I enabled auto renewal. Something went into a loop and ran for two days. It cost around \$250. That is the mistake worth correcting before you make it.”

WHAT HAPPENED

Auto-renewal stayed on while a process looped

It kept topping up and it kept spending, unattended, for two days.

WHY IT MATTERS

A runaway loop with auto top-up has no ceiling

Without auto top-up, the credit runs out and the spend simply stops.

WHAT TO DO

Disable it and verify

Find auto top-up or auto renewal, switch it off, and confirm it is off before you leave the page.

4.6 Handling the Key You Just Created

Treat the secret key like a password with a credit card attached to it.

Name the key clearly

“GenAI learnings” or similar, so you know what it is for

Copy it at creation time

The full secret is shown once — store it immediately



Keep it out of your code	Never paste it directly into a script you will share
Keep it out of screenshots	Blur or crop it before posting anything publicly
One key per purpose	Separate keys make it easy to revoke just one
Revoke anything exposed	If a key leaks, delete it and create a new one

4.7 What This One Key Unlocks

With the API key in place, every OpenAI model becomes reachable from your own code.

APP

Gen AI applications

Build real applications on top of the models rather than only chatting with them.

AUT

Automations

Wire the model into workflows that run without you in the loop.

AGT

Agents

Agents and agentic systems that carry out multi-step tasks.

SAAS

Micro-SaaS

Small products of your own, powered by the same API.

TWO PATHS, ONE CHOICE

There are two ways to use an LLM: run it locally, or call it through an API. Topics 8 to 10 cover the first. For everyday building, this is the path.

4.8 Before You Move On

Complete every line — this one matters.

\$10 credits added	On platform.openai.com, nothing more
Auto top-up disabled	Verified as off before leaving the page
Secret key created	Named, copied and stored safely
mini / nano selected	Your default models for this phase
No jumping to other providers	One key is enough for now
Purchase logged in your notes	Record the date and the amount

DO NOT SKIP THIS

The rest of the journey builds directly on top of a working API key.



05

THE CLOSED PATH

LLM Benchmarks

Reading benchmarks is a skill. Build it once and every future model release becomes easy to judge. The point is not the leaderboard — the point is choosing well.

5.1 Why You Should Care About Benchmarks

WHAT THEY TELL YOU

Capability, measured

- Every model is measured on what it can actually do
- Scientists design and validate these tests
- The numbers are how a lab proves the model performs
- They let you compare models on the same yardstick

WHY IT MATTERS TO YOU

Better output, less waste

- Pick the model whose benchmarks match your use case
- The right model performs better in your application
- Coding, image generation and reasoning are different strengths
- Reading benchmarks is itself a skill worth building

5.2 Where to Find a Model's Benchmarks

They will not arrive in your hand. You go and get them.

1	A model is released	Any model, any provider, any day.
2	Search the release	For example, “ release” in a search engine.
3	Open the release page	The lab's own announcement or release notes.
4	Read the benchmark section	Scroll to the charts and the tables.
5	Note it against your use case	Record what it means for your work.

MAKE IT A HABIT

A new model appears, you read its benchmarks the same day. That one habit keeps you current for the rest of your career.

5.3 Reading a Scorecard — Software Engineering

SWE-bench measures the skills a software engineer needs. Higher is better. A frontier model scoring in the high sixties on 100 real software tasks is a number worth pausing on — call a software engineer in and give them the same 100 tests, and they would not score it either.



5.4 The Benchmark Families

Four names cover most of what you will see on a release page.

SWE

SWE-bench

Software engineering. Every skill a developer needs, tested. Terminal work and coding in your IDE sit here.

HLE

Humanity's Last Exam

Multi-discipline reasoning at the very hardest level. Ten PhDs and a mathematician would score in single digits.

GDP

GDPval

Knowledge work. Real economic tasks are evaluated and scored into a single number.

FIN

Domain benchmarks

Financial analysis and other sector-specific tests, published alongside the headline numbers.

5.5 Aggregate Indexes — One Model, Several Axes

Aggregators such as OpenRouter publish combined indexes so you can see a model's shape, not just one score.

- A high agentic score with a lower coding score tells you where the model belongs
- Never take a single percentage as the whole answer
- Compare the shape against what your application actually needs
- Open the aggregator, open the model, read the profile

5.6 Turning Benchmarks Into Decisions

This is the whole point of the exercise.

If your job is...	Look at...	Which usually points to...
Writing and fixing code	SWE-bench and the coding index	Whichever model currently leads the coding scores
Generating images	The specialised image model line-up	A flagship image model for quality; a cheaper image API for volume
Hard multi-step reasoning	Humanity's Last Exam, agentic index	The frontier models, where these scores are highest
Everyday, high-volume work	Cost per million tokens plus latency	The mini and nano tier — the default for this phase

VALIDATE ON YOUR OWN DATA

Benchmarks point you in the right direction. Always validate on your own real use case before you commit.



Key Takeaway

FIND IT

Search the release

Open the lab's own page and scroll to the benchmarks.

READ IT

Note the scores that matter

Only against the tasks you actually care about.

USE IT

Choose, then verify

Pick the model whose strengths match your application, then test it on your own data.



06

THE TOOLKIT

Essential AI Tools

The rule for this phase is simple: do not spend money learning Gen AI tools. There are lakhs of them. A very small number is deliberately used here.

6.1 The Rule for This Whole Phase

FREE

Free versions only

Every tool in this topic has a free tier that is enough for what is being done here.

ESSENTIAL

Essential only

There are lakhs of AI tools. Use the essential ones and ignore the rest.

TIME

Time is the real cost

Do not lose days mastering a tool you will not use. Go direct, open the chat, start working.

6.2 The AI Tool Ecosystem

Everything you will encounter falls into one of these three categories.

Category	What it is for	Examples
Gen AI chat tools	Your daily assistant	ChatGPT · Gemini · Kimi · Qwen · DeepSeek · Claude · Grok
AI code editors	Where you build	Desktop: Antigravity · Cursor · Replit · Kiro · GitHub Copilot. Web: Lovable · Bolt · Emergent · Base44
Agentic tools	Autonomous task execution	Prompt in, work done. Powerful, and covered in a later phase — not needed today

WHY THE CATEGORIES MATTER

Know the three categories so that when someone mentions a tool you can place it immediately.

6.3 The Chat Tools We Actually Use

Four tools, all free. Sign in normally and start using them.

GPT

ChatGPT

The obvious starting point, and the one most people already have.

GEM

Gemini

Excellent free tier, and Pro can be activated if you have access through your account.



KIM

Kimi

Kimi Chat — free and capable. Do not go deeper into its extra features yet.

QWN

Qwen

Qwen Studio generates images, slides and artifacts. Available on mobile too.

CLAUDE AND GROK

Both are excellent but better on paid tiers — use them if you already have a subscription through work.

6.4 AI Code Editors

Two shapes: something you install, or something you open in a browser.

DESKTOP APPS

Installed on your machine

- Antigravity IDE — from Google
- Cursor — installs and behaves like VS Code
- Replit and Kiro
- GitHub Copilot
- All of these are built on top of the VS Code base

WEB APPS

Nothing to install

- Lovable, Bolt, Emergent, Base44
- Open the site and give it a prompt
- A landing page, web page or full site comes back
- Again, it is prompt craft that decides the quality

6.5 Agentic Tools

Tools that take a prompt and carry out the work themselves, rather than assisting you while you do it. They are genuinely powerful, and they are covered properly in a later phase. Know the category exists; park the tools for now.

6.6 AI-Assisted vs Agentic — The Key Distinction

One line separates everything in the tool landscape.

AI-ASSISTED TASK / CODING

This is what is being learned here

- You use AI to help you finish a task
- You stay in the loop and make the decisions
- The tool accelerates you; it does not replace you
- This is the part of the map covered right now

AGENTIC CODING / TASK

This comes later

- You give a prompt and the tool does the work itself
- Meant purely for doing development and executing tasks
- Powerful — and covered properly in a later phase
- Set this part aside for now; it is not the focus yet



6.7 Your Stack for This Phase

This is all you install and use.

Tool	Why it is on the list
VS Code	Everything is built here. Not Cursor, not Antigravity — those can wait.
Four free chat tools	ChatGPT, Gemini, Kimi and Qwen. DeepSeek if you like. Completely free — there is no payment.
OpenAI API key	The \$10 key from Topic 4. One key, and the mini or nano models.
Nothing else, for now	Cursor and Antigravity are optional. Agentic tools belong to a later phase.

WHY SO RESTRAINED

Most tools you see are wrappers, and you will be able to build them yourself. The goal is to become a system builder, not a tool user — and that is genuinely within reach.

6.8 Before You Move On

Sign in to the four chat tools	ChatGPT, Gemini, Kimi, Qwen — free accounts
Activate Gemini Pro if eligible	Free where you already have access through your account
Keep VS Code as your editor	This is where all the building happens
Do not purchase any tool	Free versions are enough for everything ahead
Recognise the three categories	Chat tools, code editors, agentic tools
Park the agentic tools	Know they exist; return to them in a later phase

WHAT UNLOCKS THESE TOOLS

Prompt craft is what unlocks all of them. That is a topic of its own, and it is coming.



07

THE TOOLKIT

Hugging Face Spaces

Someone has already built the thing you need, published it free, and left it running in your browser. This topic is about finding it.

7.1 What Hugging Face Actually Is

If social media is where people post, this is where AI companies post their models.

THE PLATFORM

Social media for AI

- Large AI companies and small teams alike publish here
- The AI products and models they build are released openly
- The model files themselves are made available
- You download them and run them on your own system

WHAT THAT MEANS

Full access, mostly free

- Around 90% of what you see is free
- About 10% are gated models offered for purchase
- Pricing is visible on the Pricing tab
- If a model is open source, it is available here

7.2 The Site Map

Type “hugging face” into your browser, open the first result, and this is the interface you land on.

Section	What it holds	Why you go there
Models	The open model catalogue	Every open-source model, complete details, files and versions, filter by size
Datasets	Training and evaluation data	Public datasets, paired with the models, free to browse
Spaces	Working applications	Live demo apps built by the community, running in your browser — the focus of this topic
More	The rest of the platform	Buckets, docs, enterprise plans and pricing

7.3 What Spaces Are

A Space is a working application that someone has built around an open-source model and left running for anyone to use. No install, no account required for most, and no cost.



7.4 How a Space Works

1	Open-source model	A model sitting in the Hugging Face catalogue.
2	Space / application	A developer wraps it into a working app.
3	Gradio or Streamlit	The Python UI framework that renders it.
4	Interface in browser	You type a prompt or upload a file.
5	Output back to you	The result appears — no install, no cost.

GRADIO OR STREAMLIT?

They differ in name more than in purpose. Both are Python UI frameworks — if you know Streamlit from Phase-004, Gradio is simple.

7.5 What You Will Find Inside Spaces

IMG

Image generation

Prompt in, image out. Many models are live and ready.

VID

Video generation

The same idea applied to video output.

TXT

Text generation

Open text models running behind a simple interface.

TRN

Language translation

Translate between languages without any setup.

SPK

Speech synthesis

Generate speech from text in the browser.

SNT

Sentiment analysis

Classic NLP tasks, performed directly inside a Space.

7.6 A Live Walkthrough — Image Generation

1	Open huggingface.co	First result in the search, then open the interface.
2	Click Spaces	This is the area that matters most for productivity.
3	Choose the Image Generation task	A large number of models appear under it.
4	Open a model	The app interface loads directly in the browser.
5	Type your prompt	Optionally add a reference image.
6	Run and collect the output	The image is generated right there, at no cost to you.



7.7 The Two Spaces Worth Knowing Today

From genuine day-to-day use, these two deliver the most value.

OCR

Getting data out of documents

- Scraping data out of a PDF is often painful
- Paste or upload into an OCR Space and it extracts the content
- The output comes back as clean Markdown
- That makes everything downstream far easier to process

IMAGE UPSCALING

Restoring damaged images

- Photographs where the pixels have degraded
- Upscaling Spaces restore and sharpen them
- Extremely useful and completely free to try
- One of the most-used Spaces in practice

7.8 Your Action for This Topic

The reason this topic exists is simple: an enormous amount of free productivity is sitting there unused.

Open Spaces and browse	Give the whole catalogue an overall look first
Try image generation	Run one prompt end to end and see the output
Try an OCR Space	Push a PDF through it and look at the Markdown you get back
Try image upscaling	Restore a low-quality photograph
Note which ones fit your work	You only find out by actually using them
Shortlist your favourites	Keep a short list you can return to



08

THE OPEN PATH

Pulling an Open-Source Model from Hugging Face

A genuine skill worth having — and one that demands respect for your machine's limits. This is method one of two for running open-source models: Hugging Face plus Python.

8.1 The Reality Check Before You Start

This is a skill worth having. It is not something to do casually on a low-spec laptop.

2–10 GB

TYPICAL MODEL SIZE

Most open models are well above 2 GB

16 GB+

RAM RECOMMENDED

Plus a good GPU for comfortable use

Real risk

OF A CRASH

Laptops have been damaged doing this

WHERE THIS BELONGS

The right machine

- A VM or server your company provides
- A high-end machine you already own
- You must know the steps for when that day comes

WHERE IT DOES NOT

The wrong machine

- A low-capacity laptop running a large model
- Running large models locally is not recommended
- Learn the method — choose the machine carefully

8.2 Two Methods to Run an Open-Source Model

Both are covered in this phase. This topic is method one.

METHOD 1 — HUGGING FACE + PYTHON

This topic

- Pull the model files directly from Hugging Face
- Load them with a Python script in VS Code
- Full control over tokeniser, model and generation
- The method you need for a VM or server

METHOD 2 — OLLAMA

Topic 9

- A tool that wraps the whole process
- Far easier to run models on your local system
- One command to pull, one command to run
- Covered in full in the next topic

8.3 The Full Flow, End to End

- | | | |
|---|------------------|--|
| 1 | Hugging Face Hub | Browse Models and pick one. |
| 2 | Select a model | Check size, downloads and the files tab. |
| 3 | Pull / download | hf download into a local directory. |



4	Local PC	Files land in ./models on your disk.
5	Model files	Weights, tokeniser and config, all local.
6	Ready to run	A Python script loads it and you chat.

LEARN THE METHOD, NOT THE MACHINE

This method stays identical wherever you do it — laptop, VM or server.

8.4 Project Setup in VS Code

Create the project, isolate the environment, install the packages.

Command Prompt – project setup

```
# 1. Create a project folder and open it in VS Code

# 2. Open Terminal > New Terminal
#   Click the dropdown and select Command Prompt
#   Do NOT use PowerShell for this

> python -m venv venv
> venv\Scripts\activate
(venv) environment is now active

> cls

# 3. Install the packages
> pip install transformers torch huggingface_hub
```

WATCH FOR

Select Command Prompt in the terminal dropdown and close PowerShell. Activate the environment before installing anything, and let every package finish installing before moving on.

8.5 Hugging Face Account and Login

You need an account before you can pull anything.

1	Sign up at huggingface.co	Email address and a password.
2	Choose a username and full name	This becomes your public profile.
3	Accept the terms	The account is created immediately.
4	Confirm the verification email	Check your inbox — and your spam folder.



5 Log in from the terminal Authenticate your local environment against the account.

6 You are ready to pull The download command will now work.

8.6 Pulling the Model

Pick a small model first. Your laptop will thank you.

Command Prompt – download the model

```
# Ask for a very small model - the laptop is a small one

> hf download Qwen/Qwen2.5-0.5B-Instruct \
  --local-dir ./models/qwen-0.5b

creating folder ./models
creating folder ./models/qwen-0.5b
downloading model files ...
done (~1 GB of model files on disk)
```

SIZING GUIDE

TinyLlama 1.1B needs roughly 4–6 GB RAM. Qwen2.5 0.5B needs roughly 2–4 GB, and the model file itself is only around 500 MB. **--local-dir** decides exactly where the files land.

8.7 The Run Script

Load the tokeniser, load the model, take input, generate output. The full script is in **opensource.py** alongside this document.

opensource.py – run the local model

```
from transformers import AutoTokenizer, AutoModelForCausalLM

MODEL_PATH = "./models/qwen-0.5b"      # local model path

tokenizer = AutoTokenizer.from_pretrained(MODEL_PATH)
model     = AutoModelForCausalLM.from_pretrained(
    MODEL_PATH, torch_dtype="auto", device_map="cpu")

while True:
    user_input = input("\nYou: ")
    messages   = [{"role": "user", "content": user_input}]
    text       = tokenizer.apply_chat_template(
        messages, tokenize=False, add_generation_prompt=True)
    inputs     = tokenizer(text, return_tensors="pt")
    outputs    = model.generate(*inputs, max_new_tokens=200)
    print("\nAssistant:", tokenizer.decode(outputs[0]))
```



- **MODEL_PATH** points at the folder you just downloaded
- The tokeniser converts your text into tokens
- The model loads its weights into memory
- **generate()** predicts the response token by token
- **decode()** turns the tokens back into readable text

8.8 What You Get, and What to Improve

WHAT YOU GAINED

A model of your own

- No API cost — nothing at all
- The model runs on your own laptop
- Run it for as long as you like
- Speed and output quality follow your system's capability

WHAT TO FIX NEXT

From hard-coded to interactive

- A hard-coded prompt means you cannot change the input
- Take the input in real time inside the loop instead
- Then wrap the same script in Streamlit for a UI
- Input in the front end, model in the back, output on screen

Troubleshooting and Good Practice

Errors will appear	Expect them, read them, solve them — that is part of the exercise
Wait for the download to finish	A partial download will not load
The laptop may stall	A large model on limited RAM can freeze the machine
Watch your RAM	This is the single most important constraint
Start with the smallest model	0.5 billion parameters is a sensible first step
Check the load messages	Tokeniser loaded, model loaded, weights loaded — then it is ready



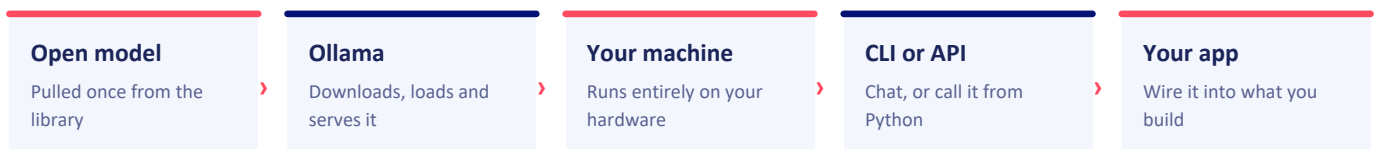
09

THE OPEN PATH

Running an Open-Source Model Using Ollama

One command to pull, one command to run. This is method two — and by far the easier of the two. It is the method you will actually reach for.

9.1 The Ollama Architecture



WHY IT EXISTS

“The easiest way to build with open models.” No manual file handling, no tokeniser or loader code to write — one command to pull, one command to run.

9.2 Install and Verify

1	Go to the Ollama site	Search for Ollama and open the official site.
2	Click Download	Pick your operating system and download the installer.
3	Run the installer	Click through the prompts and let it complete.
4	Verify the installation	Run <code>ollama --version</code> in Command Prompt, or Terminal on macOS.

9.3 Reading the Ollama Model Library

One important distinction on the models page.

LOCAL MODELS

Free to download and run

- Downloaded to your machine and run there
- No payment, no API key
- The Qwen and Gemma families are available here
- Most models sit around 5–6 GB — plan for that

CLOUD MODELS

Marked with a cloud symbol

- These are accessed rather than downloaded
- Using them requires payment
- Easy to spot by the symbol next to the name
- For learning, stay on the free local models



9.4 The Commands You Will Use

Four commands cover everything in this topic.

Command Prompt – Ollama

```
# 1. Confirm Ollama is installed
> ollama --version

# 2. Run a model (pulls it automatically the first time)
> ollama run qwen2.5:0.5b
pulling manifest ...
pulling model 397 MB
success
send a message

# 3. Chat with it
>>> Generate an email for applying for an AI engineer role
(the model responds locally)

# 4. Run it again later - no re-download
> ollama run qwen2.5:0.5b
```

- The model downloads once and is cached
- Close the terminal, reopen it, run the same command — it starts immediately
- Everything runs locally with no API cost
- Gemma 2 at around 2 GB is another good option

9.5 A Live Run

Pull a 400 MB model and chat with it. On a machine with 4 GB of RAM this still works — which is the whole point of starting at 0.5B.

9.6 Ollama App vs Command Line

THE OLLAMA APP

A chat interface

- Opens as a normal chat window
- Models load and you type straight into it
- Only a limited selection of models is offered
- Convenient for a quick try

THE COMMAND LINE

The recommended route

- Far more models are accessible
- Any model in the library can be pulled by name
- Same experience, no restrictions
- This is what is used throughout



9.7 Size vs Quality — The Honest Trade-Off

Small models are wonderfully convenient. They are also genuinely limited.

~400 MB

QWEN2.5 0.5B

Runs on almost any laptop

~600 MB

TINYLLAMA

Small, fast, and noticeably weaker output

~2 GB

GEMMA 2

A better balance if your machine allows

WHY SMALL MODELS UNDER-PERFORM

Capacity is the constraint

- Lower capacity means lower output quality
- Good thinking and strong output come from larger models
- Those were trained on far larger data and are far bigger

WHERE SMALL MODELS FIT

Still genuinely useful

- Small, well-defined tasks
- Learning and experimentation
- Machines with 4 GB of RAM, where nothing else will run

9.8 Connecting Ollama to Your Application

You can do it. You mostly should not.

HOW IT WORKS

The Python package

- Ollama publishes a Python package
- Use its chat method to talk to any model you pulled
- Point your application at the local model instead of an API
- It works, and it works well for local experimentation

THE PRODUCTION WARNING

Stated plainly

- Do not use this as the backend of a production app
- A locally-run model takes a long time to respond
- The user gets frustrated and stops paying you
- Always use an API in production

Method 1 vs Method 2 — Which to Use When

Both give you a locally-running open-source model. They differ in effort.

	Hugging Face + Python (Topic 8)	Ollama (Topic 9)
Setup	venv, packages, account, login, download command	Install the app, then one command
Code	You write the loader and generation script	None required to start chatting
Control	Full control over tokeniser and generation	Sensible defaults handled for you
Model files	You manage the folders yourself	Pulled and cached automatically
Best for	VMs and servers, and understanding the internals	Getting a model running quickly on your own machine



IF THIS IS YOUR FIRST TIME

Watch the walkthrough twice. It becomes simple very quickly.



10

THE OPEN PATH

Open-Source Model with Ollama — Hands-On Practice

The concept becomes real the moment you run it yourself. Complete this without skipping it.

10.1 Your Hands-On Task

Install Ollama, pull the Qwen2.5 0.5B model, give it an input, take an output — and capture two screenshots.

MODEL

qwen2.5 · 0.5B

0.5 billion parameters, around 400 MB on disk.

MACHINE

Almost anything

Works on virtually any laptop. 4 GB of RAM is enough.

PROOF

Two screenshots

The model being pulled, and your conversation with it.

10.2 The Hands-On Workflow

Five steps. Everything you need was covered in Topic 9.

1	Install Ollama	Download from the official site and install it.
2	Verify	ollama --version confirms the install.
3	Pull the model	ollama run qwen2.5:0.5b — about 400 MB.
4	Give it an input	Any prompt you like, typed into the chat.
5	Capture the output	Screenshot the pull and the conversation.

10.3 Deliverables and Validation

Check every line before you call it done.

Ollama installed	Verified with ollama --version
Model pulled	qwen2.5 0.5B, roughly 400 MB, downloaded successfully
Screenshot 1 — the pull	The model being pulled, showing success
Screenshot 2 — the chat	Your input and the model's response, visible together
Input and output both visible	The conversation must be legible in the screenshot
Both screenshots captured	Two screenshots — that is the whole deliverable

**DO NOT SKIP IT**

Doing it yourself is what turns the concept into practice.

10.4 Common Problems and What to Do

Symptom	Cause	What to do
The laptop stalls	The model is too large for the memory available	Stay on the 0.5B model
Download incomplete	The pull was interrupted	Let the pull finish fully before you try to run the model
Command not found	Ollama has not finished installing	Check ollama --version
Weak output	Expected on a 0.5B model	Quality follows size and your system — try Gemma 2 if RAM allows

10.5 Everything to Complete in This Phase

The Ollama hands-on task is the main deliverable. These were also set along the way.

Decode one open-source model	Capabilities, usage, context window, size — Topic 2
Decode five closed-source models	Capabilities, context, output, cost — Topic 3
Purchase the OpenAI API key	\$10, auto top-up disabled — Topic 4
Explore Hugging Face Spaces	Try image generation, OCR and upscaling — Topic 7
Run a model with Ollama	Install, pull, chat, screenshot — Topics 9 and 10
Capture the two screenshots	The pull and the conversation — the main deliverable

10.6 Phase-005 in One Picture

Ten topics, four themes. This is what you now know how to do.

Theme	Topics	What it gave you
Foundations	1–2	What an LLM is · how it works step by step · tokens and embeddings · LLM, LMM, SLM, VLM · open source vs closed
The closed path	3–5	The four major players · frontier vs specialised · token pricing · buying an API key · reading benchmarks
The toolkit	6–7	Free chat tools · AI code editors · agentic tools explained · the Hugging Face platform · Spaces for free productivity
The open path	8–10	Pulling models from Hugging Face · running them with Python · Ollama from install to chat · local vs API trade-offs · the hands-on task



Where You Stand Now

You can explain how an LLM works, choose between an open and a closed model, buy access responsibly, read a benchmark, and run a model on your own machine.

UNDERSTAND

The concepts

Tokens, embeddings, attention, and the four model families.

CHOOSE

The judgement

Open source or closed source, judged on benchmarks and on cost.

BUILD

The practice

An API key in hand, free tools installed, and a local model running.

What Is in This Phase

File	What it is
README.md	The written overview of the phase, with the topic index and the learning path
LLM-Foundations-and-Practical-Tools.pdf	This document — the full written companion to the deck
LLM-Foundations-and-Practical-Tools.pptx	The 102-slide visual walkthrough of the same ten topics
opensource.py	The Topic 8 run script — loads a local Hugging Face model and chats with it
requirements.txt	The pinned Python environment used across the phase

The Journey So Far

Phase	Focus
Phase-001-Programming-Foundations	Python fundamentals
Phase-002-GenAI-Python-Toolkit	Mandatory Python concepts for Generative AI
Phase-003-Robotic-Process-Automation-(RPA)	Desktop and browser automation
Phase-004-FastAPI-And-Streamlit	Interfaces and APIs — Streamlit and FastAPI
Phase-005-LLM-Foundations-and-Practical-Tools	This phase — LLMs, tooling and local models

BRAND PALETTE

Coral Red **#FF4A62** → **#FF5A45** for highlights and accents. Deep Blue **#031273** for structure, headings and diagrams. Black is never used as a design colour.